BSIDES SÃO PAULO · 2026

Por que seu app iOS precisa de um RASP



Quando o servidor sozinho não te salva — e o que fazer no dispositivo do usuário.

Kelvin Clark · Segurança Mobile · kclark@mobileappdefense.com

AGENDA

O que vamos cobrir em 30 min



1. O mito do 'iOS é seguro'

Por que o sistema operacional, sozinho, não protege seu app.



2. 2 casos reais em iOS

GoldPickaxe e SparkCat o que eles ensinam sobre confiança no dispositivo.



3. O que é RASP

Definição, capacidades e onde encaixa no MASVS da OWASP.



4. Como começar

Ferramentas (incl. open source), armadilhas e checklist.

MITO #1

“Mas iOS é seguro, né?”



O MITO

“App Store revisa tudo, sandbox é forte, ninguém quebra meu app.”

Conclusão errada 'eu não preciso me preocupar com o cliente'.



A REALIDADE

- 1º trojan iOS roubando dados biométricos (GoldPickaxe, 2024)
- 1º stealer de cripto dentro da App Store (SparkCat, 2025)
- Jailbreak detection é rotineiramente burlado com Frida
- Engenharia reversa de IPA é trivial sem proteção

FUNDAMENTO

O cliente é território inimigo

> *“Tudo que você envia para o dispositivo do usuário pode ser lido, modificado ou descartado por ele.”*



Binário no bolso

Atacante tem o .ipa local: pode descompactar, inspecionar e re-empacotar.



Runtime observável

Frida/objection permitem instrumentar o app rodando sem precisar do código-fonte.



Decisões no cliente

Cheques de jailbreak, biometria e regras de negócio podem ser simplesmente desligados.

ROGUES GALLERY · 2015 → 2025

iPhones nunca foram imunes

2015

XcodeGhost

Xcode malicioso infecta apps na App Store. ~128M usuários afetados (incl. WeChat).

2016+

Pegasus (NSO)

Spyware comercial zero-click via iMessage/WhatsApp. Mira jornalistas e ativistas.

2020+

LightSpy

Implante modular ligado ao APT41. Apple emitiu alerta em 92 países em 2024.

2023

Op. Triangulation

4 zero-days + features de HW não documentadas. Cadeia de 14 etapas via iMessage.

2024

GoldPickaxe.iOS

1º trojan iOS para roubo de biometria. Deepfakes para autenticar transferências.

2025

SparkCat

1º stealer dentro da App Store. OCR varre galeria por seed phrases de wallets.

Vamos aprofundar nos dois últimos — onde RASP teria mudado o jogo.

CASO REAL #1 · 2024

GoldPickaxe.iOS

1º

trojan em iOS que rouba dados biométricos de rosto

O que aconteceu

- Distribuído via TestFlight e perfis MDM, fora da App Store.
- Disfarçado de apps de governo/banco na Tailândia e Vietnã.
- Capturava foto do rosto e RG, interceptava SMS e tráfego.
- Faces eram usadas em deepfakes para autenticar transferências.
- Vítima no Vietnã: prejuízo equivalente a US$ 40.000.

LIÇÃO #1

O que faltou no app legítimo?

Atacante usou...

- Engenharia social (fingir ser app de governo)
- Distribuição fora da App Store (TestFlight/MDM)
- Captura de biometria pelo próprio fluxo do app
- Deepfake para passar pela autenticação do banco

O que RASP detectaria

- Perfil MDM/configuração suspeita instalado
- Hooks de Frida/objection no processo
- Câmera/sensor sendo gravado por outro processo
- Resposta: matar a sessão ou bloquear transferência

CASO REAL #2 · 2025

SparkCat

1º

trojan stealer descoberto dentro da App Store oficial

Como funciona

- Apps aparentemente legítimos (delivery, mensagens, IA).
- SDK malicioso embutido com modelo de OCR (Google ML Kit).
- Varre a galeria do usuário em busca de frases-semente de cripto wallets.
- Exfiltra imagens contendo as 12/24 palavras para servidor C2.
- Modelo OCR varia conforme idioma do device (latim, chinês, coreano…).

LIÇÃO

Defesa só no backend deixa um buraco



Build & Release

SAST, assinatura, App Store review



Backend

WAF, autenticação, anti-fraude



Dispositivo do usuário

??? — aqui é onde o atacante mora



O elo mais frágil hoje não é o servidor — é o app rodando em milhões de iPhones que você não controla.

CONCEITO

Afinal, o que é RASP?

RASP · Runtime Application Self-Protection

Camada de proteção que vive DENTRO do app e observa o ambiente em tempo de execução, detectando manipulação, hooking e ambientes hostis — e reagindo (avisar, degradar, encerrar).



Analogia para iniciantes

Antivírus protege o computador. WAF protege o servidor. RASP protege o app — de dentro. É o app que olha para si mesmo e pergunta: 'eu estou rodando num ambiente seguro? alguém está mexendo em mim agora?'

DEFESA EM CAMADAS

RASP não substitui — ele completa

SAST

Static App Security Testing

Olha o código antes de compilar. Encontra bugs no código-fonte.

QUANDO

Durante o desenvolvimento

DAST

Dynamic App Security Testing

Testa o app já compilado, simulando ataques. Não está em produção.

QUANDO

Antes de publicar

RASP

Runtime Self-Protection

Dentro do app em produção, no celular do usuário, agindo em tempo real.

QUANDO

Em produção, agora

CAPACIDADES

O que um RASP detecta em iOS



Jailbreak

Arquivos do Cydia/Sileo, sandbox quebrada, fork() liberado.



Hooking dinâmico

Frida, Cycript, dylibs injetadas via DYLD_INSERT_LIBRARIES.



Debugger anexado

ptrace, sysctl(KERN_PROC) detectando depuração.



Emulador / clone

Simulador iOS, app re-assinado, perfis empresariais.



Integridade

Checksum do binário, hash de Mach-O, swizzling indevido.



Rede & MITM

Proxies (Burp/Charles), certificados raiz instalados pelo usuário.

MÃO NA MASSA

Detecção de jailbreak em iOS (Swift)

JailbreakCheck.swift

```
func isJailbroken() -> Bool {
  // Procura artefatos tipicos de jailbreak
  let paths = [
    "/Applications/Cydia.app",
    "/usr/sbin/sshd",
    "/private/var/lib/apt/"
  ]
  for p in paths {
    if FileManager.default.fileExists(atPath: p) {
      return true
    }
  }
  // fork() funciona? sandbox quebrada.
  if fork() >= 0 { return true }
  return false
}
```

O que isso faz

- Procura artefatos típicos de jailbreak no sistema de arquivos.
- Tenta um fork() em iOS íntegro, isso falha.
- Resposta do app: bloquear, sair, ou só reportar.

REALIDADE

Como atacantes burlam isso em 5 minutos

bypass.js · frida -U -l bypass.js MeuBanco

```
// Hook em NSFileManager.fileExistsAtPath:
var FM = ObjC.classes.NSFileManager;
var orig = FM['- fileExistsAtPath:'];

Interceptor.attach(orig.implementation, {
  onEnter: function (args) {
    this.path = ObjC.Object(args[2]).toString();
  },
  onLeave: function (retval) {
    if (this.path.indexOf('Cydia') !== -1)
      retval.replace(0x0);  // 'nao existe'
  }
});
```

Resultado

O check ingênuo retorna 'limpo' mesmo num iPhone jailbroken.

Por isso RASP

Um produto sério mistura dezenas de checks, ofusca o próprio código de detecção e roda em camadas que o atacante não consegue hookar facilmente.

RECAP

Por que RASP é necessário, então?

Resposta em runtime

Detecta e reage no momento exato em que o ataque acontece.

Cobre o ponto cego

Onde SAST, DAST e revisão da App Store não enxergam: o device do usuário.

Não confia no cliente

Permite construir camadas que assumem que o dispositivo é hostil.

Dá telemetria

Sinaliza para o backend que aquele cliente está comprometido — antes do prejuízo.

PADRÃO

OWASP MASVS-RESILIENCE



MASVS = OWASP Mobile App Security Verification Standard. O grupo de controles que cobre RASP é o MASVS-RESILIENCE.

L1 — Baseline

Para qualquer app. Boas práticas de cripto, autenticação, comunicação.

L2 — Defense in depth

Para apps sensíveis (banco, saúde, governo). Exige proteção adicional.

R — Resilience

Para apps que enfrentam atacante motivado. Engloba RASP, hardening, anti-tampering.

QUEM PRECISA

Seu app precisa de RASP?

SIM, vale investir

- Banco, fintech, carteira de cripto
- Saúde, prontuário, telemedicina
- Governo, identidade digital
- DRM, mídia paga, jogos com economia real
- Anti-fraude / anti-abuso (delivery, marketplaces)

PROVAVELMENTE não

- App público de leitura sem login
- Protótipos / MVPs internos
- Apps onde o servidor é a única fonte de verdade e não há fraude

MERCADO

Onde estamos hoje?

60%

das organizações ainda não adotaram RASP em seus apps mobile.

Pesquisa Guardsquare (2025)

O que isso significa

- Há um gap enorme entre o que o MASVS recomenda e o que o mercado entrega.
- Atacantes sabem disso — focar em mobile dá ROI alto.
- É uma oportunidade para profissionais começando agora: segurança mobile defensiva é nicho carente.

HONESTIDADE TÉCNICA

RASP não é bala de prata

Falsos positivos atrapalham UX

Banir usuários jailbroken legítimos é caro. Tenha modo 'observar antes de bloquear'.

RASP pode ser ofuscado, não inviolável

Atacante motivado eventualmente desliga. O objetivo é elevar o custo, não impedir tudo.

Performance importa

Checks pesados em loop podem queimar bateria. Use eventos, não polling agressivo.

Sem hardening, RASP cai fácil

Combine com ofuscação, certificate pinning e proteção de strings/secrets.

MÃO NA MASSA

Como começar a estudar e aplicar

Open source para estudar

- iOS Security Suite (Swift)
- OWASP MASTG (lab + writeups)

Comercial (apps críticos)

- MAD(Mobileappdefense.com)

Para aprender atacando

- Frida + objection
- DVIA-v2 (app vulnerável para iOS)
- Corellium (simulador de iPhone)

Resumo em uma frase

Se o seu app iOS mexe com dinheiro, identidade ou dados sensíveis, ele já é um alvo — e o sandbox da Apple sozinho não te salva.

Fontes & leituras

- Group-IB (2024) — GoldPickaxe.iOS, primeiro trojan iOS roubando biometria.
- Kaspersky / Securelist (2025) — SparkCat, OCR stealer dentro da App Store.
- Kaspersky (2023) — Operation Triangulation, 4 zero-days + hardware features.
- Citizen Lab / Amnesty — Pegasus (NSO), zero-click iMessage exploits.
- ThreatFabric / The Hacker News — LightSpy iOS (APT41), alerta Apple 2024.
- MacRumors / Wikipedia — XcodeGhost (2015), 128M usuários afetados.
- OWASP MASVS — mas.owasp.org/MASVS (MASVS-RESILIENCE)

Obrigado! Perguntas → **kelvin@kclrk.com** · BSides São Paulo 2026